Python Data Structures

Module 1: Python Foundations

```
orders = ["A1", "A2"]
point = (3, 4)
stock = {"Uno": 4}
seen = {101, 102}
print(102 in seen)
```

Learning Objectives

CHOOSE

Choose the right structure.

EXPLAIN

Explain tuples vs. sets.

BUILD

Build your project's dict toolkit.

Key Terms

`list`

ordered + mutable

`tuple`

ordered + locked

`dict`

key/value pairs

`set`

unique values

Hashing

fixed fingerprint

TIP

“When do you use a `for` loop vs. a `while` loop?”

FOR

```
for score in scores:
```

WHILE

```
while count <= 3:
```

Answer like an interview, with PREP: point, reason, example, point.

MODEL ANSWER

Model Answer: PREP

POINT

`for` walks a known collection; `while` repeats until a condition fails.

REASON

`while` is for when you can't know the number of passes.

EXAMPLE

`for score in scores` visits each; `while count <= 3` keeps checking.

POINT, RESTATED

Known collection? Use `for`. Waiting on a condition? Use `while`.

Reading Recap: Pick the Right Container

Today's Reading: Python Data Structures

A food delivery app needs a different container for each kind of data.
Reopen the reading:
Pick the Right Container: Python Data Structures

The Scenario: A Food Delivery App

Every night the app tracks stops, finished orders, drivers and coupons.

Wrong container: One customer can use a first-order coupon twice.

Right container: A dict finds an order's driver in about 1 step, with 2 orders or 20,000.

READING RECAP

Key Terms and Code From the Reading

LIST: CAN CHANGE

`stops.append("Oak Rd")` adds a stop at the end.

TUPLE: NEVER CHANGES

`order[1] = "7:50 PM"` stops with a TypeError.

DICT: KEY AND VALUE

`drivers["D-5521"]` gives the value "Sam".

SET: NO REPEATS

`set()` makes an empty set. `{}` makes an empty dict.

READING CHECK

The app stores the delivery fee for each zip code. Which container fits?

A

`list`

B

`tuple`

C

`set`

D

`dict`

ANSWER KEY

Answer Key: A Fee for Each Zip Code

Correct answer: D. `dict`

Each zip code is a key and its fee is the value, so the app finds a fee by its zip code in about one step.

READING CHECK

`used` starts as `set()`, then adds "Noor", "Sam", "Noor". What does `len(used)` print?

A 2

B 3

C 1

D An error

ANSWER KEY

Answer Key: A Set Keeps Each Value Once

Correct answer: A. 2

Adding "Noor" a second time changes nothing. A set keeps each value once, so `len(used)` is 2.

Your Questions From the Reading

Turn and talk, 1 min:
One question each.

We take 2 or 3 questions now.

Bigger ones wait for today's matching slide.

Same Data, Different Rules

Lists, tuples, dictionaries, and sets each solve a different storage problem.

IN THE NEWS

One Dictionary Was Too Big for Discord's Servers



Discord cached millions of read states
Go's garbage collector scanned it all
Each scan caused lag spikes for users

What does slow-at-scale cost a company in time, money, or trust?

Source: Discord Engineering Blog

A Data Structure Is a Container With Rules



Each trades off order, uniqueness, and change.

Four Structures, Four Different Rules

LIST

ordered
changeable

TUPLE

ordered
locked

DICT

key-to-value
pairs

SET

unique
unordered

Each Structure Has a Distinct Syntax

LIST

```
["a", "b"]
```

TUPLE

```
("a", "b")
```

DICT

```
{"a": 1}
```

SET

```
{"a", "b"}
```

Hashable Means a Fixed Value Fingerprint

HASHABLE

numbers • strings • tuples
immutable, so fingerprint stays fixed

NOT HASHABLE

lists • dicts • sets
mutable, so fingerprint could change

Dict keys and set elements must be hashable.

The Speed Behind Data Structures

Picking the right structure changes how much work Python must do as your data grows.

Why Data Roles Care About Big-O

O(1)

Constant time
Speed stays roughly flat
Set / dict membership

O(n)

Linear time
Work grows with data
List membership

Open the Code-Along

Open `Module1_Lesson7_python-data-structures_STUDENT.py`
Run it after every section: `python3` + the file name

Sections, in order:

1. Lists Let You Change Anything, Anytime
   Same four ideas, now in Python, one structure at a time.
2. Once Locked In, a Tuple Can't Be Changed
3. Only Hashable Values Can Be Dict Keys
   Watching Python actually reject an unhashable key is what makes it real.
4. Sets Make Membership Checks Instant
   Timing the same lookup on a list vs. a set is where Big-O stops being abstract.
5. The Cost of Checking a List Adds Up (watch-only, projected)

Lists Let You Change Anything, Anything

lists.py

```
pending_orders = ["ORD-1001", "ORD-1002", "ORD-1003"]
pending_orders.append("ORD-1004")
pending_orders[0] = "ORD-1001-R"

print(pending_orders)
print(len(pending_orders))
print(pending_orders[-1])
```

Analogy: A list is like a playlist - ordered, and you can add, reorder, or swap songs anytime.

Once Locked In, a Tuple Can't Be Changed

tuples.py

```python
log_entry = ("2024-01-15T10:32:00Z", "ERROR", 500)
timestamp, level, status = log_entry

print(f"{level} at {timestamp}: status {status}")

log_entry[0] = "2024-01-15T10:33:00Z"
```

Analogy: A tuple is like a booked flight + hotel itinerary - once it's paid for and locked in, you don't reorder or swap pieces.

FILL IN THE BLANK

Only Hashable Values Can Be Dict Keys

dicts.py

```
product_stock = {"SKU-1001": 92, "SKU-2002": 34}
product_stock["SKU-3003"] = 8
product_stock["SKU-1001"] = 89

for sku, count in product_stock._____():
    print(f"{sku}: {count}")
```

Analogy: A dict is like a restaurant menu - dish name maps to a price, and you look up by name, not by scanning every item.

Answer: Only Hashable Values Can Be Dict Keys

dicts.py

```
product_stock = {"SKU-1001": 92, "SKU-2002": 34}
product_stock["SKU-3003"] = 8
product_stock["SKU-1001"] = 89

for sku, count in product_stock.items():
    print(f"{sku}: {count}")
```

Analogy: A dict is like a restaurant menu - dish name maps to a price, and you look up by name, not by scanning every item.

Sets Make Membership Checks Instant

sets.py

```
seen_ids = {101, 102, 103}
seen_ids.add(102)
seen_ids.add(104)

print(seen_ids)
print(103 in seen_ids)
```

Analogy: A set is like a Close Friends list - no duplicates possible, and it doesn't matter what order you added people.

The Cost of Checking a List Adds Up

slow.py

```
import time

user_id_list = list(range(2_000_000))

start = time.time()
found = 1_999_999 in user_id_list
elapsed = time.time() - start

print(f"Found: {found}")
print(f"Took {elapsed:.4f} sec to check membership")
```

CODE-ALONG DONE

Back to the Slides: Review

You just typed it and ran it. Now check that it stuck, without the code in front of you.

Watch It Run: List Scan vs. Set Lookup

Step 1 of 5

List check: is 103 equal to 101? No, next

```
seen_list = [101, 102, 103]
seen_ids = {101, 102, 103}
print(103 in seen_list)
print(103 in seen_ids)
```

OUTPUT

ANIMATED IN CLASS: watch it run in the lecture recording

A Mini Data Pipeline Uses Different Structures for Different Jobs

1

EXTRACT

List of dicts

A Mini Data Pipeline Uses Different Structures for Different Jobs

1

EXTRACT

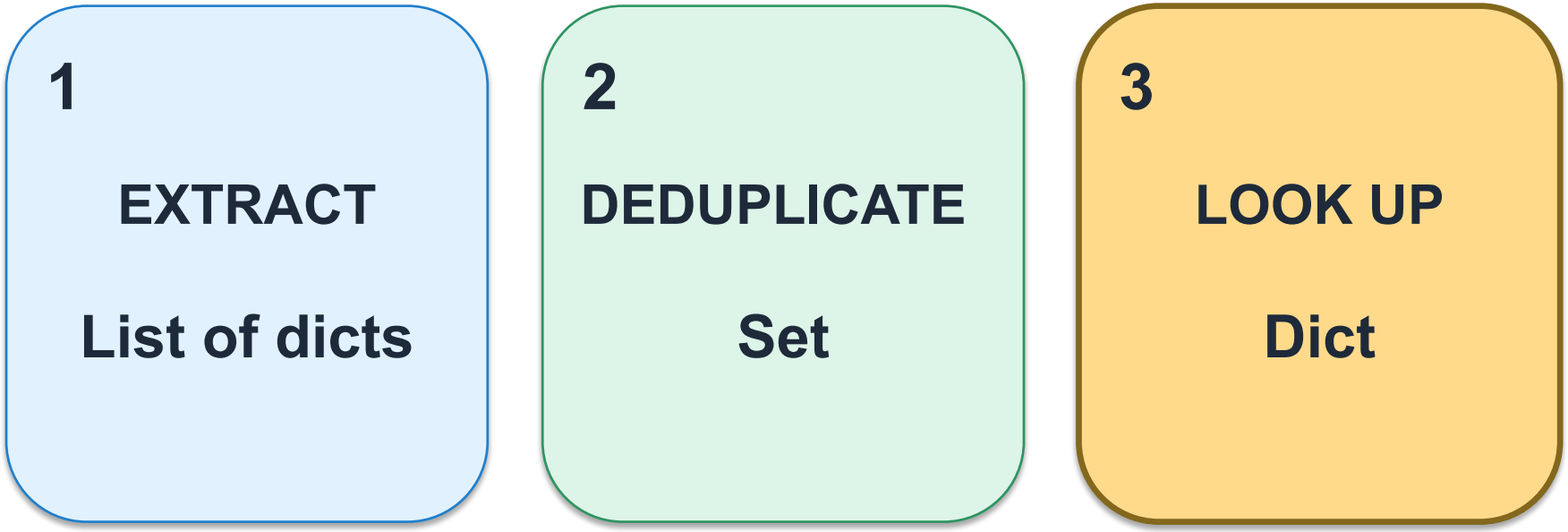
List of dicts

2

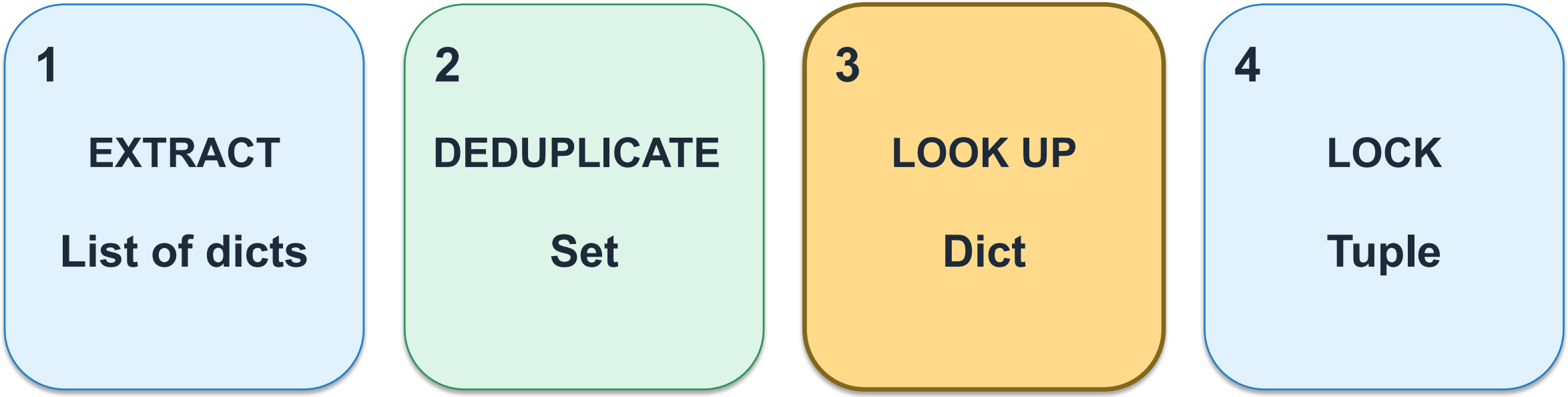
DEDUPLICATE

Set

A Mini Data Pipeline Uses Different Structures for Different Jobs



A Mini Data Pipeline Uses Different Structures for Different Jobs



TIP

“Why Lock a Record in a Tuple?”

LIST

Mutable

TUPLE

Locked

DICT

Pairs

SET

Unique

Answer like an interview, with PREP: point, reason, example, point.

MODEL ANSWER

Model Answer: PREP

POINT

Store each fixed record, like one game, as a tuple.

REASON

It can't be changed by accident after it's built.

EXAMPLE

Changing `game[1]` raises a TypeError, so the record stays as read.

POINT, RESTATED

Tuple for records that shouldn't change; list for ones I'll edit.

CHECK FOR UNDERSTANDING

Which of these four data structures is immutable?

A

`list`

B

`tuple`

C

`set`

D

`dict`

ANSWER KEY

Tuples

Correct answer: B, `tuple`

A tuple's contents can't be reassigned after creation. That immutability is also why tuples can be dict keys or set elements.

QUICK CHALLENGE

Deduplicating 5 million IDs: fastest lean choice for “seen before?”

A

`list`

B

`tuple`

C

`set`

D

`dict`

ANSWER KEY

Sets

Correct answer: `set`

Set membership is **O(1)** average case via hashing. A dict also works, but stores values you don’t need.

CONCEPTS COVERED

Interview Prep

Knowing how a set works is different from explaining why it’s faster, clearly and on the spot.

INTERVIEW PREP

Three Questions You Should Be Ready to Explain

1

List vs. set membership complexity?

2

When choose tuple over list?

3

What makes a valid dict key?

ANSWER KEY

Interview Prep: Membership

List = O(n) • Set = O(1) average case

A list may scan every item. A set uses hashing, which is why sets and dicts are the right tools for fast membership checks at scale.

ANSWER KEY

Interview Prep: Tuple vs. List

Choose a tuple when the data should not change

Examples include coordinates, fixed config values, or values you need to use safely as a dict key or set element.

ANSWER KEY

Interview Prep: Valid Dict Keys

A dict key must be hashable

Strings, numbers, and tuples work. Lists, dicts, and sets do not because their contents can change.

Key Terms — Recap

`list`

ordered + mutable

`tuple`

ordered + locked

`dict`

key/value pairs

`set`

unique values

Hashing

fixed fingerprint

So Far, We've Covered…

BEFORE

Loops & conditionals

So Far, We've Covered…

BEFORE

Loops & conditionals

TODAY

List • tuple • dict • set

So Far, We've Covered…

BEFORE

Loops & conditionals

TODAY

List • tuple • dict • set

WHY

Right structure = correct + fast

So Far, We've Covered…

BEFORE

Loops & conditionals

TODAY

List • tuple • dict • set

WHY

Right structure = correct + fast

PROJECT

Dict lookup beats linear scan

WHERE TO GO FROM HERE

Keep Learning

Python Docs: Data Structures

Official reference for lists, tuples, dicts, and sets.

Open resource

Python Tutor

Step through data structures line by line.

Open resource